

# ELI — Lora Pipeline

ELI v2.2.6

August 2026

## Contents

|                                                             |          |
|-------------------------------------------------------------|----------|
| <b>LoRA fine-tuning — audit &amp; pipeline (2026-06-07)</b> | <b>1</b> |
| What was found (audit) . . . . .                            | 1        |
| What was wired/fixed . . . . .                              | 1        |
| Still phi-profiled (recorded, your call) . . . . .          | 2        |

## LoRA fine-tuning — audit & pipeline (2026-06-07)

### What was found (audit)

The LoRA mechanisms in eli/learning/ were **individually correct but orphaned and phi-hardcoded**:

- **Algorithm — correct.** lora\_trainer.run\_training is a proper PEFT LoRA loop: tokenize (truncation/pad, labels=input\_ids) → AutoModelForCausalLM (eager attn, fp16 on CUDA, gradient checkpointing) → LoraConfig → get\_peft\_model → Trainer with DataCollatorForLanguageModeling(mlm=False) → trainer.train() → save\_pretrained (adapter + tokenizer). Safety contract is sound (dry-run default, --execute required, reviewed rows only, **GGUF never trained directly**, adapter never overwritten).
- **Stages exist** but as separate CLI scripts with no orchestrator: guard plan (lora\_trainer\_guard) → training\_preflight → dataset\_builder / export\_trainable\_dataset / merge\_reviewed\_datasets → lora\_trainer → lora\_eval. Nothing chained them.
- **Orphaned** — nothing outside eli/learning/ called any of them (no executor action, router, GUI button, scheduled task, or self-upgrade hook). ELI could not trigger or even report on LoRA.
- **Not model-agnostic** — target\_modules defaulted to phi-3's ["qkv\_proj"].

### What was wired/fixed

- **Model-agnostic target modules** (lora\_trainer.\_resolve\_target\_modules): derives LoRA targets from the LOADED architecture (scans for known projection Linear leaves: q/k/v/o\_proj, qkv\_proj, gate/up/down\_proj, c\_attn, query\_key\_value, ...), honours an explicit adapter-config override, and falls back to PEFT's architecture-agnostic "all-linear". No hardcoded architecture on the train path.
- **Pipeline DAG** (eli/learning/lora\_pipeline.py::run\_pipeline): the explicit ordered chain **preflight → build\_job → [train] → eval(inspect)**, reusing the existing modules. **Dry-run by default** (runs every gate, touches no GPU, writes no adapter — safe from chat); real training only execute=True.
- **Wired into ELI:**
  - LORA\_STATUS action (read-only): preflight readiness per target.
  - LORA\_TRAIN action: runs the pipeline DAG — **dry-run from chat**; real training only via the scheduled task / explicit GUI (execute flag not exposed in chat).

- Router: “lora status” / “is lora ready” → LORA\_STATUS; “train a lora” / “fine-tune yourself” / “run lora training” → LORA\_TRAIN.
- Scheduled lora kind (`_worker_lora`): “train a lora overnight [N steps]” → SCHEDULE\_TASK runs `run_pipeline(execute=True)` unattended.
- Manifest: 216 capabilities (includes LORA\_STATUS + LORA\_TRAIN).
- Tests: `tests/test_lora_pipeline.py` (target-module resolver, DAG order + dry-run no-train, both actions, routing, scheduled kind).

## Still phi-profiled (recorded, your call)

The training **target profiles** are still `eli_phi` / `eli_phi_ultra` (and `bootstrap_phi3_base.py`, the rope shim) — the chosen training base is Phi-3. The *algorithm* is now model-agnostic; making the **base profile** swappable (any HF causal-LM dir, not just Phi-3) is a larger, separate change — deferred to your go-ahead. LoRA needs a Hugging Face model directory, not the GGUF ELI runs for inference, so training a new brain → adapter → merge → convert-to-GGUF is the intended flow.